PS 6: Semantic Search, Chunking, & VDB Schema

Abdulla Mansoor · CIS **731** · abdulla03@ksu.edu · Track 2 (VDB) · Schema authority: **DERIVED** (INV-11)

Artifacts. Git tag `ps6-submission`; `schema/` holds the JSON Schema, the derived ontology, the `deal` contracts, the FAISS id-mapping layer, worked examples and `requirements.lock`. `./run_all.sh` reproduces every number here and exits non-zero on any failure; it currently reports **36/36 validation cases passing**. Index figures are measured on this machine (faiss-cpu 1.15.1, 2 threads); inputs that are estimates are labelled as estimates where they appear.

Chunking Cost Model

Model `sentence-transformers/all-MiniLM-L6-v2`: d = 384, max sequence length = 256 tokens, float32. Corpus N_docs = 12,000 tracks (midpoint of the PS 4 target of 10k–14k). Mean tokens per document L = 740. **L is an estimate, not a measurement**: the corpus has not been acquired and the model's tokenizer could not be downloaded in this environment, so tokens-per-word could not be counted. The basis is ~550 words per track × ~1.35 WordPiece tokens/word (slang and proper nouns fragment more than general English). A sensitivity table follows so the conclusion can be checked against a range rather than resting on the point estimate.

Derivations

Fixed window of n tokens with stride s: the first window covers [0, n) and each later window advances by s, so chunks per document is $c = \lceil (L - n)/s \rceil + 1$ for L > n, else 1. The overlap fraction is $\rho = 1 - s/n$, and overlap inflation is $1/(1-\rho) = 1/(s/n) = \mathbf{n/s}$ — the two expressions the spec names are the same quantity, which is the check that the algebra is right. Sentence-boundary chunking never splits a line, so a line that would overflow pushes the window closed early; with mean fill ratio η the effective window is ηn and $c = \lceil L/(\eta n) \rceil$. η = 0.85 corresponds to a mean line of ~0.15n ≈ 29 tokens at n = 192, about one bar. Structure-aware chunking emits one chunk per verse or hook, ~6.5 sections per track, independent of L. Index storage is N_chunks × d × 4 bytes in every case.

Chunk-length cap

Two independent ceilings apply and the smaller binds. **Model:** 256 tokens — beyond this the encoder truncates silently, so a longer chunk is half-embedded and its tail is unretrievable while still occupying an index slot. **Granularity:** 192 tokens — a trope is evidenced by a term *in context*; a chunk spanning several verses dilutes that evidence across an averaged vector, and a chunk shorter than a bar loses the context that makes the term interpretable. A verse is ~16 bars × ~12 tokens ≈ 192. The granularity ceiling binds, so **n = 192, s = 128** (ρ = 1/3, inflation 1.5), leaving 64 tokens of headroom for special tokens and bar-length variance before truncation.

Strategy	chunks /doc	ρ	1/(1−ρ)	N_chunks	Storage MiB	Token amp.
fixed-window(n=192, s=128)	6	0.333	1.50	72,000	105.5	1.56×
fixed-window(n=192, s=192)	4	0.000	1.00	48,000	70.3	1.04×
fixed-window(n=256, s=128)	5	0.500	2.00	60,000	87.9	1.73×
sentence-boundary(n=192, η=0.85)	5	0.000	1.00	60,000	87.9	1.00×
structure-aware(~6.5 sections)	7	0.000	1.00	84,000	123.0	1.00×

L (tokens/doc)	fixed(192,128)	sentence(192)	structure-aware
400	3 ch · 53 MiB	3 ch · 53 MiB	7 ch · 123 MiB
740	6 ch · 105 MiB	5 ch · 88 MiB	7 ch · 123 MiB
1100	9 ch · 158 MiB	7 ch · 123 MiB	7 ch · 123 MiB

The whole index is 70–160 MiB across the entire plausible range of L. **Storage is not a constraint at this corpus size**, so the chunking choice should be made on retrieval quality, not bytes — which inverts the usual framing of a cost model. Structure-aware chunking costs 17% more than fixed-window at L = 740 and is the only strategy whose chunk boundaries coincide with the unit a trope actually inhabits; that is worth 18 MiB. Its weakness is that it depends on section markers being present in the scraped text, which for lyrics is unreliable — the fallback is fixed(192,128).

Index and Similarity Analysis

Cosine equals inner product on unit vectors

By definition $\cos(u,v) = \langle u,v \rangle / (\|u\|\|v\|)$. If $\|u\| = \|v\| = 1$ the denominator is 1 and $\cos(u,v) = \langle u,v \rangle$. ∎ Consequently a maximum-inner-product search over unit vectors *is* a maximum-cosine search, and no separate normalization step is needed at query time provided both sides were normalized at write time. Measured on 1000 vector pairs, max $|\langle u,v \rangle - \cos(u,v)|$ = 7.45e-09 (float32 round-off).

A second consequence is used below: $\|u - v\|^2 = \|u\|^2 - 2\langle u,v \rangle + \|v\|^2 = 2 - 2\langle u,v \rangle$ for unit vectors, so L2 distance is a strictly decreasing function of inner product and **the two induce identical rankings**. This matters because HNSW's graph construction assumes a metric and inner product is not one. I expected METRIC_L2 to outperform

METRIC_INNER_PRODUCT for that reason; measured recall was identical to four decimal places at every efSearch (0.7666 / 0.8826 / 0.9442 / 0.9701 at M = 16 under both). The prediction was wrong and the equivalence above is why.

IndexFlatIP is exact because it computes ⟨q,v⟩ against every stored vector with no quantization and no pruning, then takes the top-k of the true score list: cost Θ(N·d) per query, no recall parameter to tune, and nothing to train. Measured 1.549 ms/query at N = 120,000, d = 384.

IVF versus HNSW

IVF partitions the space into nlist Voronoi cells by k-means, then scans only the nprobe cells nearest the query: query cost Θ((nlist + nprobe·N/nlist)·d), minimised near nlist ≈ √(N·nprobe); build cost is k-means, Θ(iters·N·nlist·d); memory is N·d·4 + nlist·d·4, i.e. the vectors plus a negligible centroid table. Recall degrades when a true neighbour lies in an unprobed cell — a cell-boundary effect, so it worsens as nlist rises at fixed nprobe. **HNSW** builds a navigable small-world graph and greedy-descends it, expected Θ(M·log N·d) per query with a beam of efSearch; build is Θ(N·efConstruction·M·d·log N); memory is N·d·4 plus roughly N·3M·4 bytes of links, which is additive overhead IVF does not pay. Recall degrades as efSearch narrows the beam and the descent is pruned early.

Index	recall@10	ms/query	speedup	build (s)	memory
IndexFlatIP (exact)	1.0000	1.5490	1.0×	0.12	184 MB
IVFFlat nlist=256 nprobe=1	0.9201	0.0384	40.3×	2.81	185 MB
IVFFlat nlist=256 nprobe=4	0.9999	0.1278	12.1×	2.81	185 MB
IVFFlat nlist=1024 nprobe=1	0.4057	0.0264	58.7×	15.69	186 MB
IVFFlat nlist=1024 nprobe=8	0.9985	0.0790	19.6×	15.69	186 MB
HNSW M=16 ef=64	0.9442	0.0822	18.8×	6.67	207 MB
HNSW M=32 ef=64	0.9802	0.0967	16.0×	9.27	230 MB
HNSW M=32 ef=128	0.9863	0.1263	12.3×	9.27	230 MB

Measured on 120,000 synthetic unit vectors drawn from 200 Gaussian clusters, d = 384, 1,000 queries, k = 10, ground truth from IndexFlatIP. **The data is synthetic because the corpus is not yet embedded.** Latency and memory are index properties and transfer; the recall column characterises this distribution and is indicative only. Cluster spread was set to 0.35 of the unit sphere — an earlier run that scaled noise per-coordinate rather than by 1/√d produced effectively uniform points and measured 1–30% recall for *both* families, which is the right answer for uniform high-dimensional data and a badly misleading one for an embedding space.

IVF at nlist = 256, nprobe = 4 reaches 0.9999 recall at 12× the speed of exact search, trains in 2.8 s and adds 0.4 MB over the raw vectors. HNSW at M = 32 needs 230 MB and 9.3 s to build and still returns 0.9802 at ef = 64. At this scale IVF dominates on every axis except the extreme low-latency corner. **The operative recommendation, though, is to use neither.** Exact search costs 1.55 ms/query on 120k vectors; the entire ANN apparatus buys ~1.4 ms, and §2 below shows that is 0.1% of the query budget. ANN becomes worth its tuning surface and its recall risk somewhere above ~10^6 vectors — roughly an order of magnitude more than this project will hold.

Re-ranking latency

T = T_enc + T_ann + k·T_ce, retrieving k = 50 and re-ranking to 10. T_ann is measured. T_enc = 8 ms (bi-encoder, one short query, CPU) and T_ce = 22 ms (cross-encoder, one pair, CPU) are **literature estimates** — the models could not be downloaded here, so they are not measured and are flagged as such.

Configuration	T_enc	T_ann	k·T_ce	T total	cross-enc. share
IndexFlatIP (exact)	8.0	1.549	1100	1109.5 ms	99.1%
IndexIVFFlat(nlist=256, nprobe=4)	8.0	0.128	1100	1108.1 ms	99.3%
IndexFlatIP, k=20	8.0	1.549	440	449.5 ms	97.9%
IndexFlatIP, no re-rank	8.0	1.549	0	9.5 ms	0.0%

The cross-encoder is 99% of the budget and the index is 0.01–0.14% of it. Swapping exact search for IVF changes end-to-end latency by 0.1%; dropping k from 50 to 20 more than halves it. **Any effort spent tuning nlist/nprobe before tuning k is misallocated.** One caveat on the model itself: k·T_ce assumes pairs are scored serially, which is pessimistic — batching 50 pairs through a cross-encoder amortises the fixed per-call cost and in practice lands well below 50×T_ce. The linear form is the spec's and is the right *upper* bound; the conclusion that the re-ranker dominates survives either way.

Retrieval Evaluation Protocol

Let q be a query, R_q the set of chunks genuinely relevant to it, and r_1…r_k the ranked results. **P@k** = |{r_1…r_k} ∩ R_q| / k, the share of the shown results that are relevant. **Recall@k** = |{r_1…r_k} ∩ R_q| / |R_q|, the share of all relevant chunks that surfaced. **MRR** over a query set Q is (1/|Q|)·Σ 1/rank_q, where rank_q is the position of the first relevant result (0 if none in the top k). P@k and MRR need judgements only on *returned* items; Recall@k needs |R_q|, a property of the whole corpus.

What counts as relevant

Few-shot trope retrieval needs a graded judgement, because 'from the right scene' and 'actually attests the trope' are different things and collapsing them hides the system's real failure mode — returning plausibly regional text that evidences nothing.

Grade	Criterion	Example
2 — attests	The chunk contains the trope's surface form, or for [illegible]	Query 'hybrid stage', chunk uses the term in a [illegible]
1 — contextual	Same scene, micro-genre or era, and supports interpretation, but does not itself attest the trope.	Same festival, but describes the crowd, not the trope term.
0 — not relevant	Neither. Includes topically similar text from the wrong region.	Any region, term absent and context unrelated.

P@k and Recall@k binarise at grade ≥ 1; a strict variant at grade = 2 is reported alongside, and the gap between them is the quantity of interest — it is exactly the rate at which the retriever returns atmosphere instead of evidence. MRR uses grade = 2, since the first result a generator reads should be one that can be cited. **Protocol:** judgements are pooled over the top-10 of all three systems (TF-IDF baseline, bi-encoder, bi-encoder + cross-encoder), presented blind to system identity and in randomised order; 20% of the pool is double-annotated and Cohen's κ is reported with the results rather than after adjudication; ties are adjudicated by a third pass against the written criteria, never by discussion. This protocol is the input to the MP 9 gold-standard codebook.

Which metrics are gradable, and when

Metric	Before an own gold set (INV-12)	After INV-17
P@k	Computable on own queries — needs judgements only on returned items. Comparable to the cross-system baseline only if the pool is	Unchanged.
MRR	Computable on own queries, same caveat.	Unchanged.
Recall@k	Not gradable on own data. |R_q| is unknown. Gradable as recall within pool. The pool is not the corpus.	Gradable as true recall.

The per-region Recall@5 slice promised in PS 4 therefore cannot be reported as an absolute number until the gold set exists, and even then it is recall-within-pool. Until INV-17 is satisfied the honest report is P@5 and MRR with κ, plus the attribution rate, which needs no corpus-wide denominator at all and is the metric least damaged by this limitation.

Record Schema and ID Mapping

`schema/chunk_record.schema.json` is JSON Schema Draft 2020-12, validated with jsonschema 4.26.0 as pinned in `requirements.lock` — 'the schema validates' is a claim about a specific validator, not a universal one. It extends the MP 5 CorpusChunk: MP 5 fixed record_id, source_id, the event-time bounds, text, ingest_ts and license_class; this record carries those forward and adds the embedding, the ontology linkage and the provenance digest.

Field	Type / constraint	Why it is shaped this way
chunk_id	^[0-9a-f]{64}$	SHA-256 of doc_id|chunk_index|chunker_version. Deterministic so that re-embedd
doc_id	^(genius|rym|spotify|librivox|[illegible])_.+$	Source prefix carries provenance in the key, so an unlisted source is detectable w
source_hash	SHA-256 hex	Digest of the raw document as acquired; detects upstream edits that would silentl
license_class	enum of 5	Non-admissible values are representable precisely so they can be rejected rather
corpus_type	enum of 4	Selects release_year vs broadcast_ts via if/then/else.
release_year / broadcast_ts	conditional, mutually exclusive	Draft 2020-12 if/then/else: ASR requires broadcast_ts and forbids release_year; ev
trope_ids[]	^(LEX|THM|SON|NAR)-[0-9]{4}$, unique	Key into the derived ontology. May be empty: most chunks attest nothing, and a
embedding_model / _dim / vector	enum; const; length-bound	Dim and vector length are constrained against the model by if/then, so a
token_count	1…256	The ceiling is the model's max sequence length; a larger value means the tail was
provenance_verified	boolean	Postcondition of admit_document; vdb_upsert refuses anything where it is not true

The harness runs 36 cases, including 13 deliberately malformed records — ASR with a release_year, a vector of the wrong length, a pastebin: doc_id, duplicate trope ids, an unknown extra property — each asserted to be *rejected*. All 36 pass. A schema only ever shown accepting good input demonstrates nothing.

Three properties JSON Schema structurally cannot express are listed in the schema's own `x-contract-enforced` block and enforced in code instead: unit norm (no aggregate over an array), hash integrity (shape of a digest is checkable, correctness is not), and chunk_id derivation. EmbeddingCompleteness and NoOrphanEmbeddings are set-level and cannot be stated about a single record at all.

FAISS int64 ↔ chunk_id mapping

FAISS addresses vectors by non-negative int64; chunk_id is a 256-bit digest. The bridge is faiss_id = int(chunk_id[:16], 16) & 0x7FFF_FFFF_FFFF_FFFF — the leading 64 bits masked to 63, because FAISS reserves negatives for 'not found'. Birthday collision probability at N = 72,000 is $\approx N^2/2^{64} \approx 8\times10^{-10}$: small, not zero, and a silent collision returns the wrong document with full confidence — so build_idmap *checks* rather than trusting the estimate (20,000 distinct ids, 0 collisions over 20,000 vectors). A sidecar int64 → chunk_id map carries the reverse direction.

Update and delete semantics, measured on IndexIDMap2. add_with_ids does *not* replace an existing id: re-adding one id took ntotal from 20,000 to 20,001, leaving two vectors under one key and a search free to return the stale one. The correct update is remove_ids then add_with_ids; remove_ids deletes *every* copy, so it also repairs the duplicate (ntotal back to 20,000, reconstruct returns the new vector). Delete must also drop the sidecar entry — forgetting that second step is exactly how an orphan is born. After delete: ntotal 19,999, sidecar 19,999, in bijection.

Contracts and Ontology

The two invariants, in plain language

EmbeddingCompleteness: every chunk admitted to the corpus has exactly one vector in the index — no admitted chunk is unretrievable. **NoOrphanEmbeddings:** every vector in the index belongs to exactly one live, admitted chunk — nothing is retrievable that cannot be traced to a document whose licence was checked. They are duals and both are needed: completeness alone permits a store full of unattributable vectors, NoOrphan alone permits documents admitted and then never embedded. Together they assert a bijection between the record set and the vector set.

admit_document

Precondition — license_class ∈ {public_domain, cc_by, instructor_provided}. **Postcondition** — the returned record has provenance_verified = True and a source_hash equal to the SHA-256 of the bytes actually admitted. The hash is *recomputed* inside the contract rather than trusted from the caller, because a caller-supplied digest certifies nothing.

```
@deal.pre(lambda doc, _raw: doc["license_class"] in INV9_ADMISSIBLE_LICENCES)
@deal.post(lambda result: result["provenance_verified"] is True)
@deal.ensure(lambda doc, raw_bytes, result: result["source_hash"] == sha256(raw_bytes).hexdigest())
```

vdb_upsert

Precondition — the record cleared admit_document; its chunk_id equals the derived digest; its vector length, embedding_dim and embedding_model agree; and $\|\text{vector}\|_2 = 1 \pm 10^{-5}$. **Postcondition** — the index holds exactly one vector for this chunk_id. In plain language: a chunk may be embedded as often as we like and the index will never hold two vectors for it, nor zero.

```
@deal.pre(lambda index, record: record["provenance_verified"] is True)
@deal.pre(lambda index, record: record["chunk_id"] == derive_chunk_id(…))
@deal.pre(lambda index, record: abs(l2_norm(record["vector"]) - 1.0) <= 1e-5)
@deal.ensure(lambda index, record, result: sum(1 for c in result if c == record["chunk_id"]) == 1)
```

The unit-norm precondition is not decoration: §2 proves cosine equals inner product *only* on the unit sphere, so an unnormalized vector corrupts every score it participates in while producing no error. Deletion is ordered vector-first, record-second; the reverse order leaves a window in which a vector is retrievable with no record — an orphan. Dying between the two steps in this order yields a missing embedding, which is detectable and repairable by re-embedding, rather than an unattributable vector, which is invisible without a full scan.

The harness exercises each contract from both sides: records that never cleared admission, forged chunk_ids, unnormalized vectors and both non-admissible licences are each asserted to be *refused*; a seeded completeness violation and a seeded orphan are each asserted to be *detected*. 36/36 cases pass (deal 4.24.6).

Derived ontology (CIS 731, INV-11)

Classes. Trope (abstract) → LexicalTrope | ThematicTrope | SonicTrope | NarrativeTrope; Scene (abstract) → Region | MicroGenre; plus Era, Artist, Work, Chunk. **Relations and cardinalities.** originatesIn: MicroGenre→Region 1..1 · diffusedTo: MicroGenre→Region 0..* {first_attested_year, attesting_chunk_id} · attestedIn: Trope→Chunk **1..*** · characteristicOf: Trope→MicroGenre 0..* {strength} · activeDuring: MicroGenre→Era 1..* · performedBy: Work→Artist 1..* · affiliatedWith: Artist→Region 0..* {confidence, method} · chunkOf: Chunk→Work 1..1.

Granularity. Four trope subtypes rather than one, because a single undifferentiated class cannot answer whether regional identity is carried lexically or sonically, and MP 7 needs that split as separate feature families; finer granularity — a class per word sense — would demand disambiguation unvalidatable at 12k documents. Attestation is at *chunk* level, not track level, because retrieval returns chunks: attesting at track level would break the chain between a retrieved passage and the claim it supports, and that chain is precisely what the attribution-rate metric measures. MicroGenre sits between Region and Trope because attaching tropes straight to Region conflates origin with spread and makes diffusion inexpressible. Era is *derived* from release_year rather than asserted, so an era boundary cannot contradict the record it describes.

attestedIn has minimum cardinality 1 — a trope with zero attestations is inadmissible, because it would be an assertion about the corpus the corpus does not support. This is the ontology's provenance teeth and what makes the coverage metric meaningful: a count of 1 means 'barely evidenced', not 'not yet filled in'.

Competency questions (full traversals in schema/competency_questions.md): **CQ-1** which lexical tropes originate in Memphis rap pre-2000 and are attested in Southern trap post-2010 — the question that forces originatesIn to be 1..1 while diffusedTo is 0..*; **CQ-2** which sonic and lexical tropes separate G-Funk from Boom Bap and which chunks attest each — the question that justifies the subtypes and the strength attribute; **CQ-3** for a chunk just retrieved, which tropes it attests, from which work and artist, and whether its source licence was admissible — the join to the record schema and the attribution-rate path; **CQ-4** which regions and eras fall below N attestations per trope — the coverage and fairness slice. Three questions are deliberately *out* of scope: speaker stance, artist influence (diffusedTo records co-occurrence in time, never causation), and commercial performance.

A compliance gap I am flagging rather than hiding

INV-9 limits corpus content to public-domain or instructor-provided material. The team's acquisition plan of record, from PS 4, is the authenticated Genius REST API — Tier 2, and **not** public domain. The contract above is written to the invariant rather than to the plan: admit_document refuses licensed_api and fair_use_excerpt, and the test harness asserts both refusals. Those two values exist in the enum precisely so non-conforming material can be named and rejected instead of being relabelled as something it is not. The practical consequence is that the corpus the pipeline is currently specified against would be rejected by its own ingest gate, and the team must either secure instructor-provided lyrics, substitute a public-domain corpus, or obtain an explicit exemption. I would rather surface that now than discover it at HW 12.

GenAI Audit

Yes — the Generative AI Kit was used, as Section 2 of the specification directs. **GenAI.pdf is included** and logs the five prescribed stages together with the adversarial-review findings required by Stage 5. In summary: the Kit drafted the cost-model derivations, the index trade-off matrix, the JSON Schema and the contract scaffolding, and drafted this report. I verified the arithmetic independently, and every claim about FAISS in §2 and §4 is a measurement from code in this repository rather than a quotation from documentation. Four defects were found by running the work rather than reading it — a noise-scaling error that made both index families look broken, deal validator signatures that raised TypeError instead of failing contracts, the add_with_ids duplicate-id trap, and the INV-9 licence gap above — and all four are documented in GenAI.pdf with their fixes.